

DATA 607 Wk2 Assignment 2B Marcin K

Marcin K

2026-09-11

TASK 1, Null Error Rate

- Calculate the null error rate (majority-class error rate).
- Create a plot showing the distribution of the actual class (sex).
- Explain why knowing the null error rate is important when evaluating models.

TASK 1 Approach

The first step will be determine whether males/females are the majority class, then calculate the percentage of wrong guesses if the prediction is always the most common class in a dataset.

The dataset contains three columns. Sex - truth, this is the observed reality .pred_female - soft probability, measured on a continuous interval from [0,1] .pred_class - hard prediction, with a discrete binary output [0,1], [yes,no], based on a threshold cutoff.

confusionMatrix() syntax <https://www.rdocumentation.org/packages/caret/versions/7.0-1/topics/confusionMatrix>

For the plot of the data, we are looking at the resource here from IBM developer: <https://developer.ibm.com/tutorials/awb-confusion-matrix-r/>

```
knitr::opts_chunk$set(echo = TRUE)
library(tidyverse) #contains dplyr and ggplot2
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.6.1
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(cvms)
```

```
## Warning: package 'cvms' was built under R version 4.6.1
```

```
library(ConfusionTableR)
```

```
## Warning: package 'ConfusionTableR' was built under R version 4.6.1
```

```
library(knitr)
library(ggplot2)
```

```
penguins_file_path <- file.choose() #penguins file needs to be saved on your computer.
penguins <- read.csv(penguins_file_path)
glimpse(penguins)
```

```
## Rows: 93
## Columns: 3
## $ .pred_female <dbl> 0.99217462, 0.95423945, 0.98473504, 0.18702056, 0.9947012~
## $ .pred_class <chr> "female", "female", "female", "male", "female", "female",~
## $ sex <chr> "female", "female", "female", "female", "female", "female",~
```

```
# I need to find the majority class
penguins %>% count(sex)
```

```
##      sex    n
## 1 female  39
## 2   male  54
```

```
# Here is how I am going to generate the plot, let's create two new variables ground_truth and hard_pre
```

```
penguins <- penguins %>% mutate(pred_status = case_when(
  .pred_class == "female" & sex == "female" ~ "TP",
  .pred_class == "male" & sex == "male" ~ "TN",
  .pred_class == "female" & sex == "male" ~ "FP",
  .pred_class == "male" & sex == "female" ~ "FN",
))

penguins %>% count(pred_status) #verify that mutate worked
```

```
## pred_status    n
## 1          FN    3
## 2          FP    3
## 3          TN   51
## 4          TP   36
```

```
# create confusion matrix with caret package's confusionMatrix
cm <- confusionMatrix(as.factor(penguins$.pred_class), as.factor(penguins$sex))
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction female male
##   female      36    3
##   male         3   51
##
##           Accuracy : 0.9355
##           95% CI : (0.8648, 0.976)
##   No Information Rate : 0.5806
##   P-Value [Acc > NIR] : 1.319e-14
##
##           Kappa : 0.8675
##
##   Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9231
##           Specificity : 0.9444
##   Pos Pred Value : 0.9231
##   Neg Pred Value : 0.9444
##   Prevalence : 0.4194
##   Detection Rate : 0.3871
##   Detection Prevalence : 0.4194
##   Balanced Accuracy : 0.9338
##
##   'Positive' Class : female
##
```

```
# Let's make a plot as well in ggplot
```

```
penguins %>% count(sex)
```

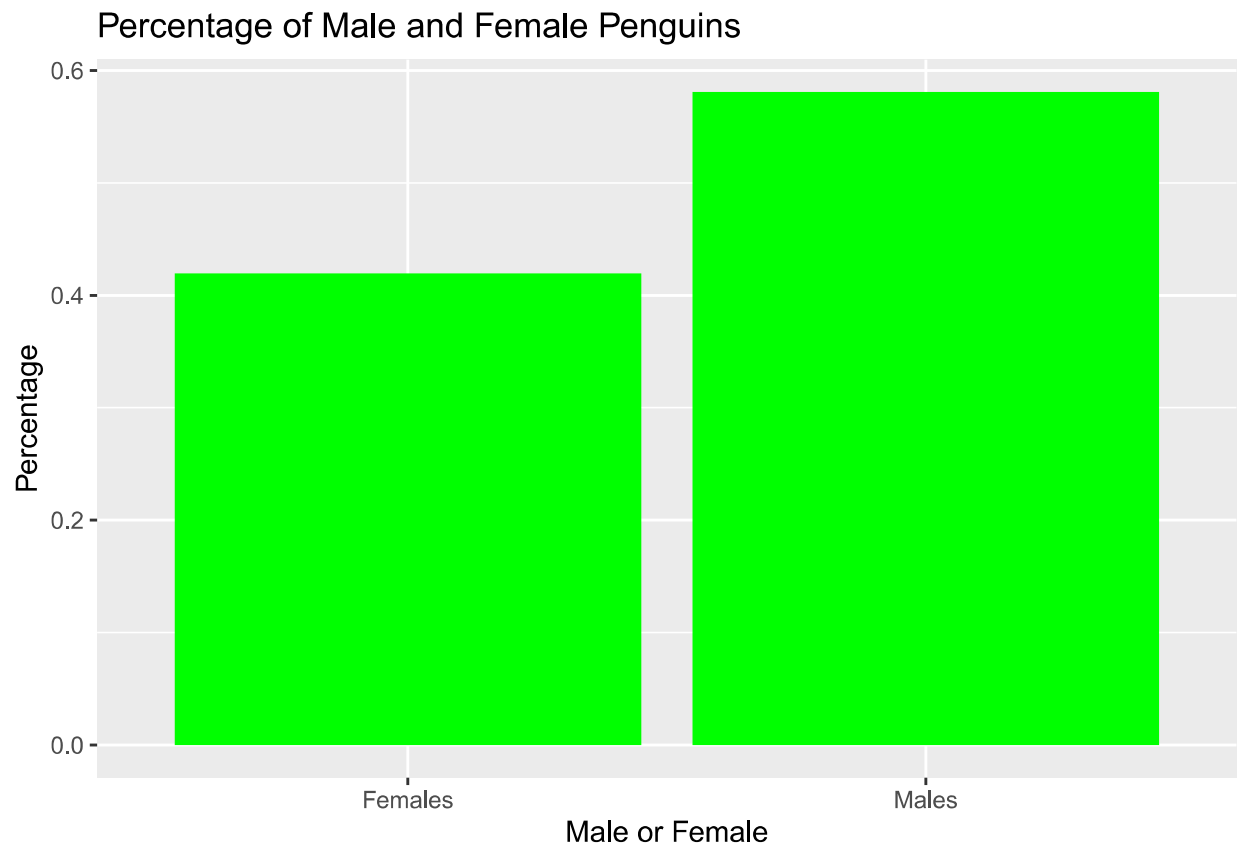
```
##      sex  n
## 1 female 39
## 2  male 54
```

```
num_males <- penguins %>% filter(sex == "male") %>% nrow()
num_females <- penguins %>% filter(sex == "female") %>% nrow()
num_total = num_females + num_males
percent_males = num_males / num_total
percent_females = num_females / num_total
```

```
data <- data.frame(category =
  c("Females", "Males"), value = c(percent_females, percent_males))
```

```
# Plotting pre-calculated numbers
```

```
ggplot(data, aes(x = category, y = value)) +
  geom_col(fill = "green") +
  labs(title = "Percentage of Male and Female Penguins", x = "Male or Female", y = "Percentage")
```



TASK 1 Answer

The majority class in the dataset is “Male”.

Found 54 males 39 females.

The majority class error rate is $(1 - \text{Majority Class} / \text{Total of Classes}) = 1 - (54/93) = 0.4193548$ or $\sim 41.9\%$

The null error rate is important to know in a comparison model because it provides the baseline error rate. It is necessary baseline to understand how effective the model is predicting the desired outcome, versus if it were simply dumb-guessing the majority class all the time. It becomes increasingly important when the dataset is imbalanced and one class dominates the other.

The confusion matrix below is the output of the confusionMatrix() function.

Confusion Matrix and Statistics

Reference

Prediction female male female 36 3 male 3 51

TASK 2, Confusion Matrices at Multiple Thresholds

- Using probability thresholds of 0.2, 0.5, and 0.8, compute: *True Positives (TP)* False Positives (FP)
True Negatives (TN) False Negatives (FN)

Present results as three confusion matrices. You may compute these manually or use existing functions.

TASK 2 Approach

To compute true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) across multiple classification thresholds, I will obtain the predicted continuous probabilities and true binary probabilities for the dataset. Then for the three thresholds listed, I will convert the prediction probability to a binary prediction by comparing the predicted probability against the threshold value. Repeat three times.

```
# Threshold of 0.8 Below

penguins <- penguins %>% mutate(.pred_class_0.8 = case_when(
  .pred_female > 0.8 ~ "female",
  .pred_female <= 0.8 ~ "male"
))

cm_0.8 <- confusionMatrix(as.factor(penguins$.pred_class_0.8), as.factor(penguins$sex))
print(cm_0.8)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction female male
##   female      36    2
##   male         3   52
##
##               Accuracy : 0.9462
##               95% CI : (0.879, 0.9823)
##   No Information Rate : 0.5806
##   P-Value [Acc > NIR] : 1.223e-15
##
##               Kappa : 0.8892
##
##  Mcnemar's Test P-Value : 1
##
##               Sensitivity : 0.9231
##               Specificity : 0.9630
##               Pos Pred Value : 0.9474
##               Neg Pred Value : 0.9455
##               Prevalence : 0.4194
##               Detection Rate : 0.3871
##   Detection Prevalence : 0.4086
##               Balanced Accuracy : 0.9430
##
```

```
##      'Positive' Class : female
##
```

```
# Threshold of 0.5 Below
```

```
penguins <- penguins %>% mutate(.pred_class_0.5 = case_when(
  .pred_female > 0.5 ~ "female",
  .pred_female <= 0.5 ~ "male"
))

cm_0.5 <- confusionMatrix(as.factor(penguins$.pred_class_0.5), as.factor(penguins$sex))
print(cm_0.5)
```

```
## Confusion Matrix and Statistics
##
##      Reference
## Prediction female male
##      female      36      3
##      male         3     51
##
##      Accuracy : 0.9355
##      95% CI : (0.8648, 0.976)
##      No Information Rate : 0.5806
##      P-Value [Acc > NIR] : 1.319e-14
##
##      Kappa : 0.8675
##
##      Mcnemar's Test P-Value : 1
##
##      Sensitivity : 0.9231
##      Specificity : 0.9444
##      Pos Pred Value : 0.9231
##      Neg Pred Value : 0.9444
##      Prevalence : 0.4194
##      Detection Rate : 0.3871
##      Detection Prevalence : 0.4194
##      Balanced Accuracy : 0.9338
##
##      'Positive' Class : female
##
```

```
# Threshold of 0.2 Below
```

```
penguins <- penguins %>% mutate(.pred_class_0.2 = case_when(
  .pred_female > 0.2 ~ "female",
  .pred_female <= 0.2 ~ "male"
))

cm_0.2 <- confusionMatrix(as.factor(penguins$.pred_class_0.2), as.factor(penguins$sex))
print(cm_0.2)
```

```
## Confusion Matrix and Statistics
```

```

##
##           Reference
## Prediction female male
##   female      37    6
##   male         2   48
##
##           Accuracy : 0.914
##           95% CI : (0.8375, 0.9621)
##   No Information Rate : 0.5806
##   P-Value [Acc > NIR] : 9.542e-13
##
##           Kappa : 0.8258
##
##   McNemar's Test P-Value : 0.2888
##
##           Sensitivity : 0.9487
##           Specificity : 0.8889
##           Pos Pred Value : 0.8605
##           Neg Pred Value : 0.9600
##           Prevalence : 0.4194
##           Detection Rate : 0.3978
##   Detection Prevalence : 0.4624
##           Balanced Accuracy : 0.9188
##
##           'Positive' Class : female
##

```

TASK 2 Answer

Below is the Confusion Matrix for the 0.8 Threshold

Reference

Prediction female male female 36 2 male 3 52

Below is the Confusion Matrix for the 0.5 Threshold

Reference

Prediction female male female 36 3 male 3 51

Below is the Confusion Matrix for the 0.2 Threshold

Prediction female male female 37 6 male 2 48

TASK 3, Performance Metrics

- For each threshold, calculate:
- Accuracy
- Precision
- Recall
- F1 score
- Present results in a clear table.

- TASK 3 Approach -

I will complete the calculation for each threshold.

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ Precision = $TP / (TP + FP)$ Recall = $TP / (TP + FN)$
F1 Score = $2 * (Precision * Recall) / (Precision + Recall)$

I will research how to best present this data in a table format. One option is the modelsummary package, but this is an opportunity to see what packages and options exist to present the data.

```
# Manual calculation of TP, FP, TN, FN for all three thresholds below

penguins <- penguins %>% mutate(.pred_status_0.8 = case_when(
  .pred_class_0.8 == "female" & sex == "female" ~ "TP",
  .pred_class_0.8 == "male" & sex == "male" ~ "TN",
  .pred_class_0.8 == "female" & sex == "male" ~ "FP",
  .pred_class_0.8 == "male" & sex == "female" ~ "FN",
))

penguins <- penguins %>% mutate(.pred_status_0.5 = case_when(
  .pred_class_0.5 == "female" & sex == "female" ~ "TP",
  .pred_class_0.5 == "male" & sex == "male" ~ "TN",
  .pred_class_0.5 == "female" & sex == "male" ~ "FP",
  .pred_class_0.5 == "male" & sex == "female" ~ "FN",
))

penguins <- penguins %>% mutate(.pred_status_0.2 = case_when(
  .pred_class_0.2 == "female" & sex == "female" ~ "TP",
  .pred_class_0.2 == "male" & sex == "male" ~ "TN",
  .pred_class_0.2 == "female" & sex == "male" ~ "FP",
  .pred_class_0.2 == "male" & sex == "female" ~ "FN",
))

# Manual calculation of accuracy, precision, F1 score, and recall

TP_0.8 <- penguins %>% filter(.pred_status_0.8 == "TP") %>% nrow()
FP_0.8 <- penguins %>% filter(.pred_status_0.8 == "FP") %>% nrow()
FN_0.8 <- penguins %>% filter(.pred_status_0.8 == "FN") %>% nrow()
TN_0.8 <- penguins %>% filter(.pred_status_0.8 == "TN") %>% nrow()

Accuracy_0.8 = (TP_0.8 + TN_0.8) / (TP_0.8 + TN_0.8 + FP_0.8 + FN_0.8)
Precision_0.8 = TP_0.8 / (TP_0.8 + FP_0.8)
Recall_0.8 = TP_0.8 / (TP_0.8 + FN_0.8)
F1_Score_0.8 = 2 * (Precision_0.8 * Recall_0.8) / (Precision_0.8 + Recall_0.8)

TP_0.5 <- penguins %>% filter(.pred_status_0.5 == "TP") %>% nrow()
FP_0.5 <- penguins %>% filter(.pred_status_0.5 == "FP") %>% nrow()
FN_0.5 <- penguins %>% filter(.pred_status_0.5 == "FN") %>% nrow()
TN_0.5 <- penguins %>% filter(.pred_status_0.5 == "TN") %>% nrow()

Accuracy_0.5 = (TP_0.5 + TN_0.5) / (TP_0.5 + TN_0.5 + FP_0.5 + FN_0.5)
```

```

Precision_0.5 = TP_0.5 / (TP_0.5 + FP_0.5)
Recall_0.5 = TP_0.5 / (TP_0.5 + FN_0.5)
F1_Score_0.5 = 2 * (Precision_0.5 * Recall_0.5) / (Precision_0.5 + Recall_0.5)

TP_0.2 <- penguins %>% filter(.pred_status_0.2 == "TP") %>% nrow()
FP_0.2 <- penguins %>% filter(.pred_status_0.2 == "FP") %>% nrow()
FN_0.2 <- penguins %>% filter(.pred_status_0.2 == "FN") %>% nrow()
TN_0.2 <- penguins %>% filter(.pred_status_0.2 == "TN") %>% nrow()

Accuracy_0.2 = (TP_0.2 + TN_0.2) / (TP_0.2 + TN_0.2 + FP_0.2 + FN_0.2)
Precision_0.2 = TP_0.2 / (TP_0.2 + FP_0.2)
Recall_0.2 = TP_0.2 / (TP_0.2 + FN_0.2)
F1_Score_0.2 = 2 * (Precision_0.2 * Recall_0.2) / (Precision_0.2 + Recall_0.2)

# Now let's neatly try and write all this information into a clean summary table. Created individual ve

Row_Names = c("0.2_Threshold", "0.5_Threshold", "0.8_Threshold")
Accuracy = c(Accuracy_0.2, Accuracy_0.5, Accuracy_0.8)
Precision <- c(Precision_0.2, Precision_0.5, Precision_0.8)
Recall <- c(Recall_0.2, Recall_0.5, Recall_0.8)
F1_Score <- c(F1_Score_0.2, F1_Score_0.5, F1_Score_0.8)

# The tibble() function allows us to join this into a summary

Summary <- tibble(Threshold = Row_Names, Accuracy = Accuracy, Precision = Precision, Recall = Recall, F1 = F1_Score)

# rm() to clear dataframe, was helpful in troubleshooting a typo

# Below is the table summary using the kable() function from the Knitr package

```

TASK 3 Answer

Threshold	Accuracy	Precision	Recall	F1
0.2_Threshold	0.9139785	0.8604651	0.9487179	0.9024390
0.5_Threshold	0.9354839	0.9230769	0.9230769	0.9230769
0.8_Threshold	0.9462366	0.9473684	0.9230769	0.9350649

TASK 4, Performance Use Cases

- Describe at least one real-world scenario where:
- A 0.2 threshold is preferable
- A 0.8 threshold is preferable

TASK 4 Approach

I will try and split this work up with a classmate. One of us will do the 0.2 model summary and another will do the 0.8 model summary. Fingers crossed that a classmate reaches out to me.

Note 9/13/26, no classmate responded in slack so we're on our own.

TASK 4 Answer

Some reading resources - <https://developers.google.com/machine-learning/crash-course/classification/thresholding> (<https://github.com/acatlin/data/blob/master/Performance%20Metrics%20for%20Classification%20problems%20in%20Machine%20Learning.pdf>)

From our the Github paper, Performance Metrics for Classification problems in Machine Learning, we would want to alter to high and low thresholds when the consequences of misclassification for either a false positive or false negative heavily outweigh the other.

For example, if our target variable is 1. The patient has cancer and 0: The patient does not have cancer, we would want a 0.2 threshold (or lower) to minimize false negatives. In the example of screening for cancer, we wouldn't want to misclassify not even a single person as it would result in a very poor outcome for a patient to leave the hospital with the mistaken assumption that they are cancer free when in fact they are not.

On the contrary, if we want to implement an aggressive spam email screening method that actively cleans the user inbox, we could set our target variable is 1. The email is spam 0: The email is not spam, we would want a 0.8 threshold (or higher) to minimize false positives. In the example our priority is a clean user inbox, and the emails are always available in the spam folder so there is little risk even if the email is misclassified. In this example the priority is a clean user inbox.

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.